
DS N° 2 INFORMATIQUE III

Calculatrice et documents non autorisés

Le langage utilisé doit être le langage C obligatoirement

Attention à la robustesse du programme

Vous pouvez définir autant de fonctions que vous le souhaitez dans chaque exercice.

Il n'y a pas nécessairement l'équivalence : une question $\Leftrightarrow$ une fonction.

Dans tous les exercices suivants, nous considérons un arbre binaire représenté par la structure suivante en langage C :

```
typedef struct Arbre {  
    int data;                // Valeur entière stockée dans le noeud  
    struct Arbre* fg;        // Pointeur vers le fils gauche  
    struct Arbre* fd;        // Pointeur vers le fils droit  
} Arbre;
```

Exercice 1 (Opérations sur des ABR) (6 pts)

1. Ecrire la fonction d'insertion d'un élément dans un ABR
2. Ecrire une fonction qui prend deux ABR en paramètre et qui va créer et retourner l'adresse d'un troisième arbre comportant toutes les valeurs des deux premiers (les éléments peuvent être ajoutés dans n'importe quel ordre tant que l'arbre final contient bien toutes les valeurs des deux arbres d'origine).

Exercice 2 (Facteurs d'équilibre d'un AVL) (6 pts)

Soit la structure d'un arbre binaire de type AVL suivante :

```
typedef struct avl {  
    int data;                // Valeur entière stockée dans le noeud  
    struct avl* fg;          // Pointeur vers le fils gauche  
    struct avl* fd;          // Pointeur vers le fils droit  
    int equilibre;           // facteur d'équilibre de ce noeud  
} AVL;
```

Dans la fonction de rotation simple droite d'un arbre de type AVL il existe 2 lignes de code qui mettent à jour le facteur d'équilibre du noeud racine de la rotation, et celui du noeud pivot. Ci-dessous se trouve le code partiel de cette fonction, avec ces formules mises en évidence :

```
AVL* rotationSimpleDroite(AVL* racine){  
    ...  
    AVL* pivot = ...  
    ...  
    eqRac = racine->equilibre;  
    eqPiv = pivot->equilibre;  
    ...  
    // Mise a jour du facteur d'équilibre de la racine  
    racine->equilibre = eqRac - min(eqPiv, 0) + 1;  
    // Mise a jour du facteur d'équilibre du pivot  
    pivot->equilibre = max( eqRac + 2, eqRac + eqPiv + 2, eqPiv + 1 );  
    ...  
}
```

Démontrer que ces formules effectuent correctement la modification des facteurs d'équilibre de la racine et du pivot pour une rotation simple droite, et ceci dans tous les cas de figure.

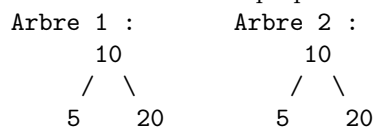
Pour cette démonstration vous pouvez utiliser ce qui vous semble le plus pertinent pour la compréhension du lecteur : phrases, schémas, ...

Exercice 3 (*Arbre identiques ?*) (5 pts)

Écrivez une fonction `sontIdentiques(Arbre * a1, Arbre * a2)`; qui vérifie si deux arbres binaires classiques sont identiques. Deux arbres sont identiques s'ils ont la même structure et les mêmes valeurs à chaque nœud. Cette fonction retournera 1 si les arbres sont identiques, 0 sinon.

Exemple :

- Pour les deux arbres suivants qui possèdent la même topologie (forme globale) :



Le résultat est 1 (ils sont identiques).

- Si un des arbres est :



Le résultat est 0 (ils ne sont pas identiques).

- Bien entendu, si les 2 arbres n'ont pas la même topologie, ils ne sont pas identiques. Le résultat est 0.

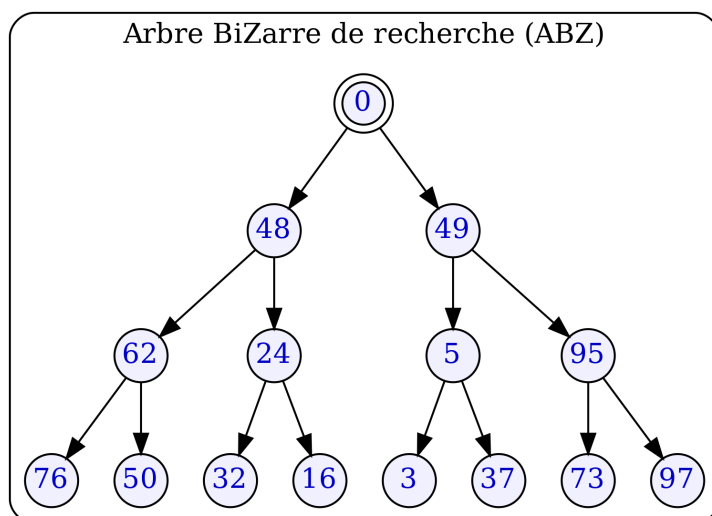
Exercice 4 (*Arbre BiZarre de recherche - ABZ*) (4 pts)

Soit un Arbre BiZarre de recherche (ou ABZ) qui possède les propriétés suivantes :

- un ABZ est un arbre binaire
- la structure d'un nœud d'un ABZ est strictement identique à celle d'un ABR (on utilisera la structure `Arbre`)
- un ABZ ne contient que des entiers positifs ou nuls : aucune valeur négative
- un ABZ ne contient aucun doublon
- un ABZ dit 'vide' contient toujours au moins un nœud racine dont la valeur vaut toujours '0'. Il ne peut pas exister d'ABZ totalement vide : il faut toujours le nœud racine avec la valeur 0.
- les valeurs paires sont stockées dans le sous arbre gauche de la racine
- les valeurs impaires sont stockées dans le sous arbre droit de la racine
- le sous-arbre des valeurs **paires** est un ABR avec les valeurs triées dans le sens décroissant (les valeurs plus grandes sont stockées dans le sous-arbre gauche, les valeurs plus faibles sont stockées dans le sous arbre droit)
- le sous-arbre des valeurs impaires est un ABR classique

De fait un parcours infixe d'un arbre de type ABZ affiche d'abord les valeurs **PAIRES** décroissantes, puis affiche la racine qui vaut 0, puis affiche les valeurs **IMPAIRES** croissantes.

Ci-dessous se trouve un exemple d'un arbre de type ABZ :



1. Écrire une fonction de recherche de valeur dans un ABZ (on utilisera le type `Arbre` déjà déclaré au début de ce document).